Code guide: the fitting layer, chunk by chunk

C. H. Ross. 2026-11-05, revised 2026-11-12. Kept in the package repository; not part of the built package.

1. Scope

This chapter walks through the code that takes a detection flatfile and returns a ranked set of fitted detection functions. It follows the order in which `sweep_models()` runs, and stops at each chunk of code to state what the chunk does, what it reads and writes, and why it is written the way it is.

It covers the fitting path (`prepare_distance_data()`, `model_set()`, `sweep_models()` and the selection table), the pre-flight checks (`diagnose_sweep()`, `detection_structure()`), the plotting method (`effect_estimates.ddf()`), and the `g(0)` slot (`availability()`, the three view-window functions, `g0()`). The fitting itself is done by `mrds::ddf()` and is described at its interface only. Effort, segmentation and the derivation of distances belong to `distsamp` and are described where they reach this package.

Line references are to the source as of the revision date (`R/sweep.R:111`). Code excerpts are abridged to the lines that carry the logic; the file and line given with each are where the full code is. The architecture and the list of guards are in `docs/01-plan.md`; this chapter does not repeat the reasoning there, and explains the code as it now stands.

Terms used throughout:

Term	Meaning
flatfile	One row per detection, with `distance` or `distbegin`/`distend`. The shape `distsamp::detection_data()` returns.
exact	A point distance in `distance`. Fitted with the exact-distance likelihood.
binned	An interval in `distbegin`/`distend`. Fitted with the binned (multinomial) likelihood over `breaks`.
truncation	The right truncation width `w`. One value per sweep.
`left`	The left truncation distance: the start of the strip the fit describes. Zero when not set.
candidate	One row of a `model_set()`: a key, an optional adjustment series and order, and a scale formula.
sweep	Every candidate in a model set fitted to the same prepared data. A `dsfit_sweep`.
`p`	Average detection probability over the searched strip, `summary(ddf)$average.p`.
`esw`	Effective strip half-width, `p * (w - left)`.
`distance_source`	The `distsamp` column naming which record supplied a distance: `angle`, `exact`, `strip`, `circling` or `break_off`.

2. The pipeline at a glance

Step	Function	File	Produces
0	`detection_structure()` (optional)	`R/structure.R:58`	what the data can support, by question
0	`diagnose_sweep()` (optional)	`R/diagnose.R:42`	a printed report; nothing fitted
1	`check_mrds()`	`R/sweep.R:378`	an error if `mrds` is absent
2	`model_set()`	`R/model-set.R:85`	one row per candidate, with its `dsmodel` string

Step	Function	File	Produces
3	`prepare_distance_data()`	`R/prepare.R:66`	`data`, `binned`, `breaks`, `n_dropped`, `dropped`
3a	`check_circling()`	`R/prepare.R:243`	an error or warning for circling-derived distances
3b	`derive_breaks()`	`R/prepare.R:290`	cutpoints, for binned data without `breaks`
4	`mrds::ddf()` per candidate	`R/sweep.R:147`	`fits`, named by `model_id`; `NULL` for a failure
5	`build_selection_table()`	`R/sweep.R:223`	the table, sorted by AIC
5a	`fit_summary()`	`R/sweep.R:257`	`n_par`, `p`, `p_se`, `esw`, goodness of fit
5b	`fit_gof()`	`R/sweep.R:303`	`cvm_p`, `chisq_p`
6	`report_sweep()`, `print.dsfit_sweep()`	`R/sweep.R:317`, `R/print.R:2`	the message and the printed summary
7	`effect_estimates.ddf()`	`R/effect-estimates.R:106`	`g(x)` on a grid, with a delta-method band
8	`availability()`, `view_window*()`, `g0()`	`R/availability.R`, `R/g0.R`	a `dsfit_g0` for the abundance step

Steps 0 and 8 sit outside the fitting path. Step 0 runs the same guards as step 3 without fitting. Step 8 builds the correction the fit deliberately does not contain. The intended route is therefore:

```
flat <- distsamp::detection_data(segs, area = 5811)
diagnose_sweep(flat, model_set(c("hn", "hr", "gamma")), truncation = 2000)
sw   <- sweep_models(flat, model_set(c("hn", "hr", "gamma")), truncation = 2000)
selection_table(sw)
```

3. `prepare_distance_data()` — `R/prepare.R:66`

3.1 Argument checks and the distance type

```
has_point <- "distance" %in% names(data) && any(!is.na(data$distance))
has_bin   <- all(c("distbegin", "distend") %in% names(data)) &&
  any(!is.na(data$distbegin))
```

The type is decided by which columns carry values, not by which columns exist, because a `distsamp` flatfile always carries all three. Neither present is an error. Both present is an error: the exact and binned likelihoods differ, so AICs from the two cannot be ranked together, and a table holding both marks a survey-era boundary. The message names `distance_source` as the column to split on.

`truncation` must be finite and positive, and `left` finite, non-negative and smaller than `truncation`. `distance`, `distbegin` and `distend` must each be numeric (or entirely missing): a character column would be compared with the truncation as text, and a factor as level codes. Negative values are an error, since a perpendicular distance has no side. Zero rows, and distance columns that are present but entirely missing, each have their own message.

3.2 Circling-derived distances — `check_circling()`, `R/prepare.R:243`

```
src    <- as.character(data$distance_source)
fitted <- !is.na(data$distance)
n_break <- sum(fitted & src %in% "break_off")   # error
n_circ  <- sum(fitted & src %in% "circling")    # warning
```

`distsamp::detection_data()` excludes detections made while circling unless `include_circling = TRUE`. When they arrive here they are one of two kinds, and the two are treated differently:

`distance_source`	What the distance is	Behaviour
`break_off`	Great-circle distance from where the aircraft left the line to where the record was logged. Not perpendicular to anything.	error
`circling`	The sighting's exact position projected onto the census line at the break-off. Perpendicular, but fixed minutes after detection.	warning, with the count

Only rows that would be fitted are counted. A circling row with no `distance` is dropped in Section 3.3 with the other missing distances. `%in%` is used rather than == so that `NA` sources compare as false. The check does nothing when `distance_source` is absent, so a flatfile from another source is unaffected.

3.3 Attrition by reason

```
gone_missing <- is.na(data$distance)
gone_beyond  <- !gone_missing & data$distance > truncation
gone_inside  <- !gone_missing & !gone_beyond & data$distance < left
```

Each dropped row is assigned to exactly one reason, in this order, so the three counts in `dropped` sum to `n_dropped`. The breakdown is returned because a total alone cannot distinguish a truncation that trimmed a tail from one that discarded half the survey. `diagnose_sweep()` prints it.

3.4 The binned branch

1. Rows with a missing `distbegin` or `distend` are removed.
2. Any `distend` of `Inf` is an error. Every `STRIP` code book has an open top bin, which no detection function can be fitted to.
3. Rows with `distend > truncation` or `distbegin < left` are removed. A bin is kept or dropped whole; it is never split.
4. `breaks` are derived (Section 3.6) if not given. Supplied `breaks` are checked by `check_breaks()` (`R/prepare.R:317`): they must be strictly increasing and contain every recorded bin edge. The likelihood uses each row's own `distbegin`/`distend`, but goodness of fit uses `breaks`, so cutpoints across the recorded bins left AIC unchanged and computed the chi-square over bins nobody surveyed.
5. The top break must equal `truncation` (`R/prepare.R:173`).
6. The bottom break must equal `left`, or zero without it (`R/prepare.R:188`).
7. The `distance` column is removed. A flatfile carries it, all `NA`, on binned rows, and `mrds` read it as the distances: every candidate failed with "no data to analyze".

Steps 5 and 6 are the same guard at the two ends of the strip. A binned fit integrates the detection function over the bins, and `mrds` normalises it over `(left, width)`. A strip between the outermost break and either limit is effort with no bin to place detections in. The bottom guard is reached in two ways: derived breaks when the innermost bin had no detections, and a `left` that falls inside a bin, whose detections step 3 then drops. Both messages name the two remedies: move the limit to the break, or supply `breaks` that reach it.

3.5 The exact branch

```
keep <- !is.na(data$distance) & data$distance <= truncation
if (!is.null(left)) keep <- keep & data$distance >= left
```

Truncation is inclusive at both ends. An empty result is an error rather than an empty fit. Empty `distbegin`/`distend` columns are removed, since `mrds` otherwise warns once per candidate that the data has bins. `breaks` given with exact distances is ignored, with a warning. `ensure_object()` (`R/prepare.R:337`) adds a sequential `object` column when absent, and refuses repeated values: `mrds` keys detections on it and rejects duplicates with a message that begins with a newline.

3.6 `derive_breaks()` — `R/prepare.R:290`

The unique `(distbegin, distend)` pairs are sorted by `distbegin`, and each pair's end must equal the next pair's start. A gap or an overlap means the intervals define no single set of cutpoints, and is an error naming the first interval before the gap. The cutpoints are the sorted unique edges.

Edges are compared with a relative tolerance of 1e-8 (`near()`), and each recorded edge is snapped to its representative before bins are compared. Bins converted between units otherwise failed to tile against themselves: `0.1 * 3` and `0.3` are different doubles.

4. `model_set()` — `R/model-set.R:85`

4.1 Validation and the grid

Keys are checked against `hn`, `hr`, `unif`, `gamma`; adjustment series against `cos`, `herm`, `poly`; factors are taken as character. `order` must be whole numbers. `formula` is a one-sided formula or a list of them; a two-sided one is refused. `expand.grid()` (`R/model-set.R:124`) crosses key, adjustment, order and formula index.

Two uniform-key candidates are removed with a warning. With no adjustment (`R/model-set.R:136`) it is a flat function, a strip transect, and fitting it as a candidate is almost never what was meant. With a covariate formula (`R/model-set.R:152`) the formula has nothing to act on: covariates enter through the scale parameter, which the uniform key lacks, and `mrds` ignores the formula rather than refusing it. Kept, the candidate would duplicate the uniform model without covariates under a covariate label.

Candidates with the same `model_id` (repeated keys, orders or formulas) are dropped with a warning after the grid is built. Kept, `fits` could not tell them apart and each would sit at `delta_aic` 0 beside its twin.

4.2 The `dsmodel` string — `build_dsmodel()`, `R/model-set.R:224`

```
fn <- if (covariate) "mcds" else "cds"
parts <- c(paste0("key = \"", key, "\""), paste0("formula = ", formula))
if (!is.na(adjustment)) {
  parts <- c(parts, paste0("adj.series = \"", adjustment, "\""),
             paste0("adj.order = ", format_orders(order)))
}
paste0("~", fn, "(", paste(parts, collapse = ", "), ")")
```

A formula other than ~1 selects `mcds`. The string is stored in the table and passed to `stats::as.formula()` at fit time, so the model a row describes is readable without refitting. `label_model()` builds `model_id` as `key[+adjustment(orders)][ covariates]`, for example `hn+cos(2,3) bf`.

4.3 Cumulative adjustment orders — `adjustment_orders()`, `R/model-set.R:254`

```
if (adjustment == "cos") { start <- if (key == "unif") 1 else 2; step <- 1 }
else                     { start <- if (key == "unif") 2 else 4; step <- 2 }
if (is.na(max)) return(start)
if (max < start || (max - start) %% step != 0) return(NULL)
seq.int(start, max, by = step)
```

Each element of `order` is the highest order of one candidate, and the candidate carries every term of its series up to it, as `Distance::ds()` does (its internal `get_adj_orders()`). `order = 3` on `hn` with `"cos"` gives `adj.order = c(2, 3)`. With no `order`, each candidate has one term, the first of its series. The gamma key takes the `hn`/`hr` series; `Distance` does not offer it. An order outside the series is dropped with a warning naming it, rather than passed to `mrds`, where it failed.

Key	Cosine	Hermite, polynomial
`hn`, `hr`, `gamma`	2, 3, 4, …	4, 6, 8, …
`unif`	1, 2, 3, …	2, 4, 6, …

5. `sweep_models()` — `R/sweep.R:111`

5.1 Arguments and settings

`truncation` has no default and its absence is an error naming why: AIC compares models fitted to the same data, and truncation defines that data. The gamma key together with `left` warns, since both describe a blind spot beneath the platform.

```
meta <- list(width = truncation)
if (!is.null(left)) meta$left <- left
if (prepared$binned) { meta$binned <- TRUE; meta$breaks <- prepared$breaks }
```

The same width, left truncation and bins are passed to every candidate, which is what makes the AICs comparable. `models` is checked by `check_model_set()` (`R/sweep.R:366`): it must have the columns `model_set()` returns, and at least one row.

5.2 Monotonicity — `monotonicity()`, `R/sweep.R:350`

```
constrain <- !is.na(model$adjustment) && model$key != "gamma" &&
  identical(trimws(model$formula), "~1")
list(mono = constrain, mono.strict = constrain)
```

Added to each candidate's `meta.data`. This is `Distance::ds()`'s default: a strict monotonicity constraint for a model with adjustment terms and no covariates, none otherwise. Unconstrained, adjustment terms can bend g(x) above 1 or back upward: `hr+cos(2)` on one simulated survey reached 1.09, with AIC 3399.27 against 3401.50 from `ds()`. Constrained, the AIC and `p` match `ds()` exactly. A key alone is monotone already. The gamma key is excluded because it rises to a peak by design.

5.3 The fitting loop — `R/sweep.R:147`

```
fit <- try(mrds::ddf(dsmodel = stats::as.formula(models$dsmodel[i]),
                     data = prepared$data, method = "ds",
                     meta.data = c(meta, monotonicity(models[i, ]))),
           silent = TRUE)
if (inherits(fit, "try-error")) {
  failures <- c(failures, setNames(
    first_line(conditionMessage(attr(fit, "condition"))),
    models$model_id[i]))
  fits[i] <- list(NULL)
} else if (!identical(as.numeric(fit$ds$converge), 0)) {
  failures <- c(failures, setNames("did not converge", models$model_id[i]))
  fits[i] <- list(NULL)
} else {
  fits[[i]] <- fit
}
```

A candidate fails in one of two ways. `mrds` either throws, or warns and returns an object with `ds$converge != 0` whose summary has no `average.p`. Both are recorded in `settings$failures` under the model's ID, and both leave a `NULL` in `fits`. `fits[i] <- list(NULL)` is used rather than `fits[[i]] <- NULL`, which would delete the element, shorten the list, and move every later fit onto the wrong row of the table.

`first_line()` (`R/sweep.R:358`) keeps the first non-empty line of an `mrds` error. Some begin with a newline, and keeping the text before it gave a blank reason.

`mrds` draws random starting values. Whether a marginal candidate converges can therefore depend on the session's random state; a seed set before the sweep makes it repeat.

5.4 The selection table — `build_selection_table()`, `R/sweep.R:223`

One row per candidate, failed or not. `converged` is `!is.null(fit)`, which after Section 5.3 means converged. `vapply()` carries the names of `fits` onto every column, and they are removed with `unname()` so a column pulled out is a bare vector. `delta_aic` is `NA` throughout when every candidate failed (`R/sweep.R:245`), rather than `NA - Inf` with a warning from `min()`. Rows are ordered by AIC with failures last.

5.5 Per-fit quantities — `fit_summary()`, `R/sweep.R:257`

```
left <- if (is.null(s$left)) 0 else s$left
p    <- scalar(s$average.p)
esw  <- p * (scalar(s$width) - left)
```

`average.p` is the mean of `g(x)` over `(left, width)`, so the strip it scales is `width - left`. `scalar()` (`R/sweep.R:286`) returns `NA` for a field that `mrds` left empty or absent, so a single malformed summary cannot stop the table being built. `p_cv` is `p_se / p`, computed in the table.

5.6 Goodness of fit — `fit_gof()`, `R/sweep.R:303`

`mrds::ddf.gof(f, qq = FALSE)` is called once per fit. `cvm_p` is taken from `dsgof$CvM$p`, which `mrds` computes for exact distances only. `chisq_p` is taken from `chisquare$chi1$p`, which exists for both. For binned data the chi-square is over the survey's own bins; for exact data it is over cutpoints `mrds` chooses, and `cvm_p` is the test to read. A chi-square with no degrees of freedom left is `NA`, which is a converged model with no test available.

5.7 Reporting

`report_sweep()` sends one `rlang::inform()` message: fits out of candidates, detections, truncation, rows dropped, the IDs that did not fit, and the `g(0) = 1` assumption. `print.dsfit_sweep()` shows the top five converged rows with whichever goodness-of-fit column applies: chi-square when every `cvm_p` shown is `NA` and some `chisq_p` is not, CvM otherwise.

6. Pre-flight checks

6.1 `detection_structure()` — `R/structure.R:58`

Reports, per question, whether the table supports it: `TRUE`, `FALSE`, or `NA` where the structure is present but incomplete. Covariate candidates are the columns that vary and are not structural. The structural list is the `mrds` and flatfile columns (`object`, `distance`, `distbegin`, `distend`, `size`, `distance_source`, and the like) plus, when `narwcr` is installed, `narwcr::narwc_never_fill()`. Leg-level columns from `narwcr::narwc_fill_columns()` are labelled as survey conditions. Aliases from `narwcr::narwc_schema()` map two columns naming one variable to a canonical name, and a duplicate is reported; a `<NAME>_ORIGINAL` column kept by `narwcr::read_narwc(prefer_source = TRUE)` is excluded. Names are compared in upper case, since NARWC's are upper case and a flatfile's often are not: `latitude` is LATITUDE, and `BEAUFORT` beside `beaufort` is one variable named twice.

`perception_structure()` (`R/structure.R:237`) requires `observer` and `detected`, at least two observers, and an `object` that appears twice. Availability is always reported as unsupported: it cannot be estimated from distances.

6.2 `diagnose_sweep()` — `R/diagnose.R:42`

Runs in order: toolchain, `detection_structure()`, `prepare_distance_data()`, sample size, model set, blind spot, covariates. Each finding is printed as `ok`, `note`, `WARN` or `FAIL`; `WARN` and `FAIL` set the closing line to "Problems above". A guard error stops the run and returns what was reached. Guard warnings, such as the circling warning of Section 3.2, are caught with `withCallingHandlers()` and printed as `WARN`

lines rather than escaping as R warnings after the report. A table with no distances at all is a `FAIL` line, not an `ok`. `models` is checked as in Section 5.1.

The truncation-loss warning (over half lost) counts rows beyond the truncation or inside `left` against the rows that have a distance. Counted against all rows, a flatfile's samples with no detection read as truncation loss, and the warning blamed the units.

The blind-spot check fires when the nearest detection exceeds 2% of the truncation width. `onset_shape()` (`R/diagnose.R:293`) then compares the first of 20 equal bins above that distance with the modal bin; a ratio above 0.6 is classed as an abrupt edge, which `left` removes and no key function reproduces.

7. `effect_estimates.ddf()` — `R/effect-estimates.R:106`

7.1 Arguments

`interval` and `scale` are resolved by `resolve_choice()` (`R/effect-estimates.R:229`) rather than `match.arg()`, because `fancyfx::plotEffects()` forwards its own unresolved default vectors. `"auto"` becomes `"se"`; `"cri"` is refused. `var` must be `"distance"`. The grid runs from `left` (or 0) to `width`.

7.2 The averaged curve — `averaged_g()`, `R/effect-estimates.R:190`

```
design <- as.data.frame(dm)
if (!is.null(shape_dm)) design <- cbind(design, as.data.frame(shape_dm))
key   <- do.call(paste, c(unname(design), sep = "\r"))
rows  <- rep(match(unique(key), key), each = length(x))
obj$scale$dm <- dm[rows, , drop = FALSE]
obj$shape$dm <- shape_dm[rows, , drop = FALSE]
xs <- rep(x, times = n_u)
```

`mrds::detfct()` pairs distances with design-matrix rows element by element, recycling the shorter. Every (distance, covariate row) pair is therefore made explicit: the unique rows of the combined scale and shape design matrices are each repeated once per grid point, and the distances are repeated once per unique row. The result is reshaped to a grid-by-row matrix and averaged with weights equal to each row's frequency. An intercept-only model has one unique row. The shape matrix, which hazard-rate and gamma keys carry, is expanded with the scale matrix so the two stay the same length. The uniform key has neither matrix, and is evaluated once on the grid; the expansion previously produced an empty grid and every value was `NaN`.

`covariate_object()` (`R/effect-estimates.R:243`) rebuilds the scale design matrix from the model's data with the `at` values substituted, so named covariates are fixed and the rest are averaged as observed. `at_value()` (`R/effect-estimates.R:272`) puts a value for a factor or character covariate on the observed levels, so `at = list(plat = "B")` works and an unknown level is refused by name; a one-level character column made `model.matrix()` fail with a message about contrasts.

7.3 The band — `delta_se()`, `R/effect-estimates.R:289`

The Jacobian of the averaged curve with respect to the parameters is taken by `numDeriv::jacobian()` and combined with `solve(model$hessian)`. The band is clamped to `[0, 1]` but never across the estimate, so it contains the curve even where an unconstrained fit puts g(x) above 1. Where the Hessian does not invert, the standard errors are `NA` and a warning is given, so the curve is still returned.

8. The g(0) slot

8.1 availability() — R/availability.R:137

Implements Laake et al. (1997):

$$a = \frac{E(s) + E(d)\left(1 - e^{-w/E(d)}\right)}{E(s) + E(d)}$$

per row, with surface, dive and window each of length 1 or the longest length; other lengths are an error rather than recycled by position. The standard error is the delta method on (surface, dive) with their covariance matrix, by numDeriv::grad() in availability_se() (R/availability.R:562). It is NA when no standard errors are given. Giving one of se_surface and se_dive without the other is an error. The result has the component shape key, component, value, se.

8.2 View windows

Function	Geometry	w(x)
view_window() (R/availability.R:275)	circular patch of radius r	2 sqrt(r^2 − x^2) / v
view_window_aerial() (R/availability.R:363)	side-window wedge, half-angle theta forward and aft	alpha + 2 x tan(theta) / v, or alpha + slope ∗ x
view_window_vessel() (R/availability.R:490)	forward sector of half-angle theta from the bow	(sqrt(r^2 − x^2) − x / tan(theta)) / v, floored at 0

view_window_ship() is an alias for view_window_vessel(). In the vessel case the animal enters the view where it crosses the search radius and leaves where its bearing exceeds theta; at theta = 90 it leaves when abeam, and the window is half of view_window(). The 90-degree case is set to an exact zero offset because tan(pi/2) is finite in floating point. The offset is computed element by element after recycling angle and distance; an ifelse() on angle previously reused the first distance for all of them.

In the aerial case a point at distance x enters the wedge x tan(theta) ahead of abeam and leaves the same distance behind, hence the factor of 2. Ganley et al. (2019) fitted the linear form empirically and did not define an angle; their 0.03 s per metre at 51.4 m/s is theta of about 37.6 degrees.

8.3 g0() — R/g0.R:112

Component tables are stacked, with source added as NA when absent. The following are errors: a component with no rows, no name, an empty-string key, a non-numeric value or se, a missing or non-finite se, a value outside (0, 1], several unkeyed rows of one component, and a component whose keys differ from the others (R/g0.R:201). A missing availability or perception component is a warning. Per key (R/g0.R:247), the value is the product of the matching components and the CV is the root sum of squared component CVs, which is the delta method under independence. An unkeyed single-row component applies to every key.

9. Ordering constraints and invariants

Constraint	Where enforced	Consequence if broken
One truncation, `left` and set of bins for every candidate	`sweep_models()`, `R/sweep.R:136`	AICs from different data are ranked together.
Adjustment models constrained monotone, as in `Distance`	`monotonicity()`	g(x) above 1; AIC and `p` differ from `ds()`.
Guards before fitting	`prepare_distance_data()` called before the loop	`mrds` errors on mixed or open-ended data, or fits it silently.
Failed fits leave a `NULL` in place	`fits[i] <- list(NULL)`	Later models take the previous model's fit and row.
Unconverged fits are failures	`R/sweep.R:166`	The table cannot be built; previously stopped the sweep.
`esw` uses `width - left`	`fit_summary()`	`esw`, and density, overstated by `left / (width - left)`.
Binned breaks span `(left, width)` exactly	`R/prepare.R:173`, `R/prepare.R:188`	Unaccounted effort at either end of the strip.
Shape and scale design matrices expanded together	`averaged_g()`	Hazard-rate and gamma curves fail or recycle.
Circling checked on fitted rows only	`check_circling()`	Rows dropped for no distance are reported twice.

10. Defects found and corrected

10.1 Second audit, 2026-11-12

Three independent auditors (statistical correctness; R code and edge cases; documentation and tests against the code) reviewed the package. Findings reported by more than one are marked †.

Defect	Correction
† `view_window_vessel()` reused the first distance's arc term for every distance when `angle < 90`: 200, 199, 196, 173 s where 200, 179, 156, 73 s is correct. The roxygen example showed it.	Offset computed element by element (Section 8.2).
† A binned flatfile with an all-`NA` `distance` column failed every candidate with "no data to analyze".	The column is removed in the binned branch (Section 3.4).
Adjustment models were fitted without the monotonicity constraint `Distance::ds()` applies, so g(x) could exceed 1 and AIC and `p` differed from `ds()`.	`monotonicity()` (Section 5.2). The adjustment snapshot changed; `unif+cos(1,2)` and `unif+cos(1,2,3)` now match `ds()` at 2178.68 and 2180.49.
`effect_estimates_ddf()` returned `NaN` throughout for every uniform-key model.	Uniform key evaluated directly (Section 7.2).
`view_window_aerial(angle =)` used `x tan(theta) / v` for a view documented as `theta` forward and aft, half the geometric window. The example's 57 degrees matched 0.03 s/m only because of it.	`2 x tan(theta) / v`; example angle 37.6 degrees (Section 8.2).
† `effect_estimates_ddf(at =)` failed for a factor covariate given by level.	`at_value()` (Section 7.2).
† Supplied `breaks` were checked only at their ends: unsorted or misaligned breaks fitted, with `chisq_p` computed over the wrong bins (0 against 0.585 on one test).	`check_breaks()` (Section 3.4).
† `diagnose_sweep()` counted a flatfile's empty samples as truncation loss and warned about units.	Counted against rows with a distance (Section 6.2).

Defect	Correction
Failure reasons were blank for `mrds` errors beginning with a newline.	`first_line()` (Section 5.3).
Duplicated `object` IDs failed every candidate with that blank reason.	Refused in `ensure_object()` (Section 3.5).
Character, factor and negative distances were accepted; character ones were compared with the truncation as text.	Refused (Section 3.1).
Bins converted between units failed to tile, with a self-contradictory message ("ends at 0.3 and the next begins at 0.3").	Tolerant comparison and snapping (Section 3.6).
NARWC vocabulary was matched by case: `latitude` offered as a covariate, `beaufort` not recognised as a condition, case duplicates not flagged.	Compared in upper case (Section 6.1).
`availability()` recycled lengths that do not divide, pairing values by position.	Lengths must be 1 or the longest.
`model_set()` kept duplicate candidates and accepted two-sided formulas.	Duplicates dropped with a warning; two-sided formulas refused (Section 4.1).
`g0()` crashed on a zero-row component and accepted character values, `se = Inf` and empty keys; several unkeyed rows gave a misleading message.	Each refused by name (Section 8.3).
`sweep_models()` and `diagnose_sweep()` crashed on a non-model-set `models`; `truncation = Inf` and an empty model set were accepted.	`check_model_set()`; finite truncation (Sections 3.1, 5.1).
The effect band could lie below the estimate where g(x) exceeded 1.	Band never crosses the estimate (Section 7.3).
The availability vignette's code was not run, and its printed output was stale: rows counts that contradicted each other, an undefined dataset, an object that errors as built, an old print layout, a broken link.	Code chunks run; output generated at build.
Documentation: the curve and `average.p` were said to differ by "a fraction of a percent" (3.5% on the package's fixture); `selection_table()`, `availability()` and package help misstated their return or scope; `NEWS.md` described a test that no longer exists; `docs/01-plan.md` gave a stale test count and an AIC claim the snapshot contradicts; data help called Ganley et al. both paywalled and CC-BY.	Each corrected.
Tests that could not fail: a blind-spot test matching text the code never prints; a failure test asserting only `is.logical()`; a covariance test asserting only inequality; an `mcds` snapshot that never checked `mcds`.	Each asserts the behaviour now.

10.2 First audit, 2026-11-04

Defect	Correction
A candidate that `mrds` returned unconverged stopped the whole sweep: its summary has `average.p` of length 0, and `vapply()` in the table builder errored. Such a fit was also labelled `converged`.	Recorded as a failure, "did not converge", when `ds$converge != 0`. `scalar()` makes empty summary fields `NA`. `delta_aic` no longer calls `min()` on an all-`NA` vector. Tests in `tests/testthat/test-sweep.R`.

Defect	Correction
`effect_estimates_ddf()` failed for intercept-only hazard-rate and gamma fits ("non-conformable arguments") and recycled for their covariate fits: only the scale design matrix was expanded to the grid.	Shape design matrix expanded alongside it. Tests check that the curve integrates to `average.p` for both keys.
`esw` was `p * width` under left truncation. At `left = 30`, `truncation = 400`, the half-normal `esw` was 118.4; the correct value is 109.5.	`p * (width - left)`. The left-truncation snapshot updated; AIC, `p` and ranking unchanged.
Binned data whose lowest break exceeded `left` (or zero) was fitted over `(left, width)` with no bin covering the inner strip. A `left` inside a bin dropped that bin's detections silently.	Refused, with both remedies named. Tests in `tests/testthat/test-prepare.R`.
The bins-do-not-tile error named interval `nrow(bins) - sum(gaps)`: "interval 4" for a gap after interval 2.	Names `which(gaps)[1]`.
Circling-derived distances were fitted without comment, and `distance_source` was offered as a detection covariate.	`check_circling()` (Section 3.2); `distance_source` added to the structural columns.
A uniform key with a covariate formula was accepted, and `mrds` ignored the formula: `unif+cos1 bf` and `unif+cos1` had the same parameter count (1) and AIC (1633.22).	Dropped with a warning by `model_set()`, as the uniform key with no adjustment already was. Test in `tests/testthat/test-model-set.R`.
Adjustment orders were one term per candidate: `order = 3` fitted the order-3 cosine term alone, unlike `Distance::ds()`. Orders outside a key's series (`hn+herm2`, `unif+herm3`) were passed to `mrds` and failed.	Cumulative, by `adjustment_orders()` (Section 4.3); out-of-series orders dropped with a warning. On the adjustment snapshot `unif+cos` with orders 1 and 2 has AIC 2178.35 against 2379.98 for the order-2 term alone.
The comment on `delta_se()` said it returned zeros when the Hessian does not invert. It returns `NA`.	Comment corrected.

11. Limitations

- `onset_shape()` uses `distbegin` for binned data, where a 20-bin histogram of bin starts carries little information about the onset. Its verdict is meaningful for exact distances only.
- Convergence of marginal candidates depends on the random state (Section 5.2). The sweep does not set a seed.
- `prepare_distance_data()` refuses a flatfile carrying both `distance` and `distbegin`/`distend` values, including one in which `distance` is the interval midpoint.
- A binned row with `distance_source == "break_off"` is not refused, since `check_circling()` counts rows with a `distance`. `distsamp` is not known to produce one.
- `at =` rebuilds the scale design matrix only. A model with a shape formula would ignore `at` for the shape; `model_set()` never builds one.
- The vignettes and tests use synthetic data. No real NARWC extract was swept in this audit.

12. Verification performed

Check	Result
`R CMD check --no-manual`, before fixes	0 errors, 0 warnings, 1 note (top-level pkgdown files); all tests pass

Check	Result
Unconverged candidate, before fix	`hr` on three distances: `ds$converge = 1`; `fit_summary()` returned `p = numeric(0)`; the sweep stopped in `vapply()`
Hazard-rate and gamma curves, before fix	Both stopped with "non-conformable arguments"; half-normal correct (mean 0.3509 against `average.p` 0.3494 on a 100-point grid)
`esw` under `left = 50`, `truncation = 400`, before fix	104.9 reported; `p * (width - left)` = 91.8
Bins 100–400 with no `left`, before fix	Fitted with `left = 0`, `int.range = (0, 400)`
`R CMD check --no-manual`, after the first audit	0 errors, 0 warnings, 1 note (unchanged); 446 expectations pass, 0 fail, 0 test warnings
Monotone `hr+cos(2)` against `Distance::ds()` defaults	AIC 3401.498 both; `p` 0.4570 both
`R CMD check --no-manual`, after the second audit	0 errors, 0 warnings, 0 notes; 508 expectations pass, 0 fail, 0 skipped, 0 test warnings; both vignettes build
`hn+cos(2,3)` against `Distance::ds(order = c(2, 3))`	AIC 2179.705 both; 3 parameters
Uniform key with a covariate, before fix	`unif+cos1 bf` and `unif+cos1`: 1 parameter each, AIC 1633.22 each

13. Planned work

1. Sweep a real extract end to end, from `distsamp::detection_data()`.

14. References

Buckland, S.T., Anderson, D.R., Burnham, K.P., Laake, J.L., Borchers, D.L. and Thomas, L. (2001) *Introduction to Distance Sampling: Estimating Abundance of Biological Populations.* Oxford University Press. Truncation, sample size, goodness of fit, and effective strip half-width.

Miller, D.L., Rexstad, E., Thomas, L., Marshall, L. and Laake, J.L. (2019) Distance sampling in R. *Journal of Statistical Software* 89(1):1–28. doi:10.18637/jss.v089.i01. The `mrds` interface.

Becker, E.A. and Quang, P.X. (2009) A gamma-shaped detection function for line-transect surveys with mark-recapture and covariate data. *Journal of Agricultural, Biological and Environmental Statistics* 14:207–223. doi:10.1198/jabes.2009.0013. The gamma key.

Laake, J.L., Calambokidis, J., Osmek, S.D. and Rugh, D.J. (1997) Probability of detecting harbor porpoise from aerial surveys: estimating g(0). *The Journal of Wildlife Management* 61:63–75. doi:10.2307/3802415. The availability formula of Section 8.1.

Laake, J.L. and Borchers, D.L. (2004) Methods for incomplete detection at distance zero. In S.T. Buckland et al. (eds) *Advanced Distance Sampling*, pp. 108–189. Oxford University Press. Why the `g(0)` components are separate.

Ganley, L.C., Brault, S. and Mayo, C.A. (2019) What we see is not what there is: estimating North Atlantic right whale *Eubalaena glacialis* local abundance. *Endangered Species Research* 38:101–113. doi:10.3354/esr00938. The aerial view window of Section 8.2.